020 最小闭环

2026年9月3日 17:45

一、三模块闭环

1 UI 对话模块

- 使用 Qt Widgets 构建简单的聊天窗口。
- 支持多行输入、发送按钮、聊天记录显示。
- 预留流式输出接口（可以先不做流式，等基本跑通后再加）。

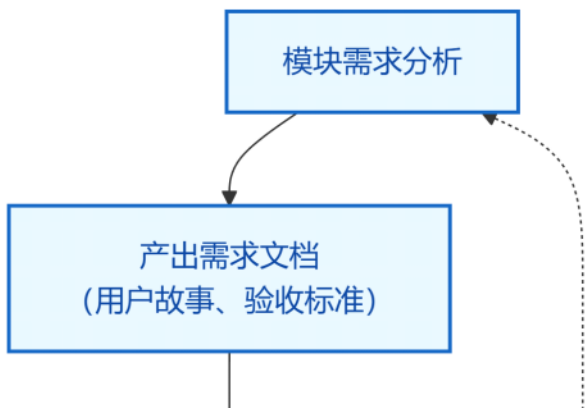
2.2 多模型适配模块

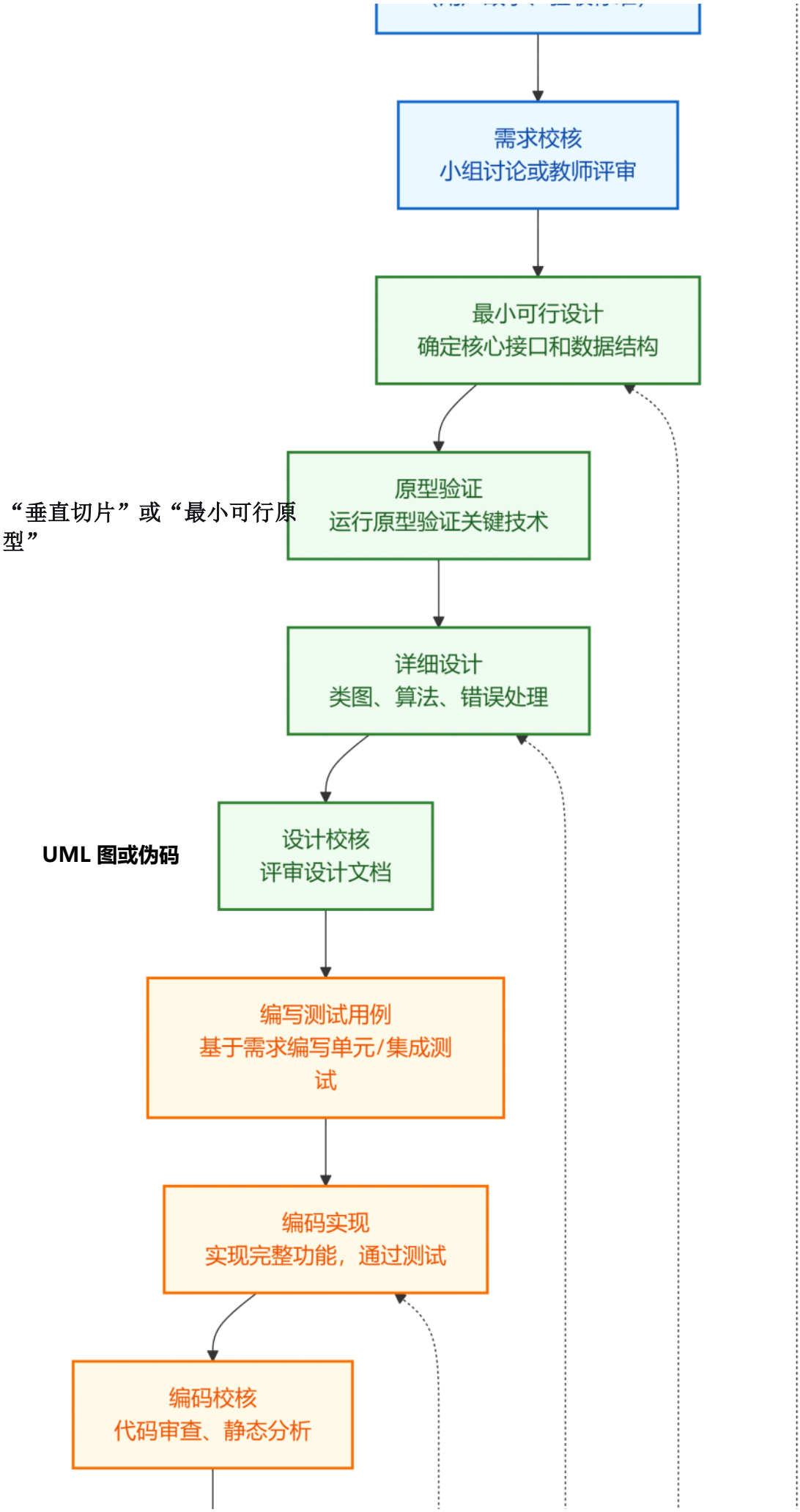
- 定义 IChatModel 接口，包含 chat(messages) 方法。
- 实现一个 MockModel 用于开发测试，避免前期依赖真实 API。
- 实现至少一个真实模型适配器（如 OpenAI 或 DeepSeek），验证接口可行性。

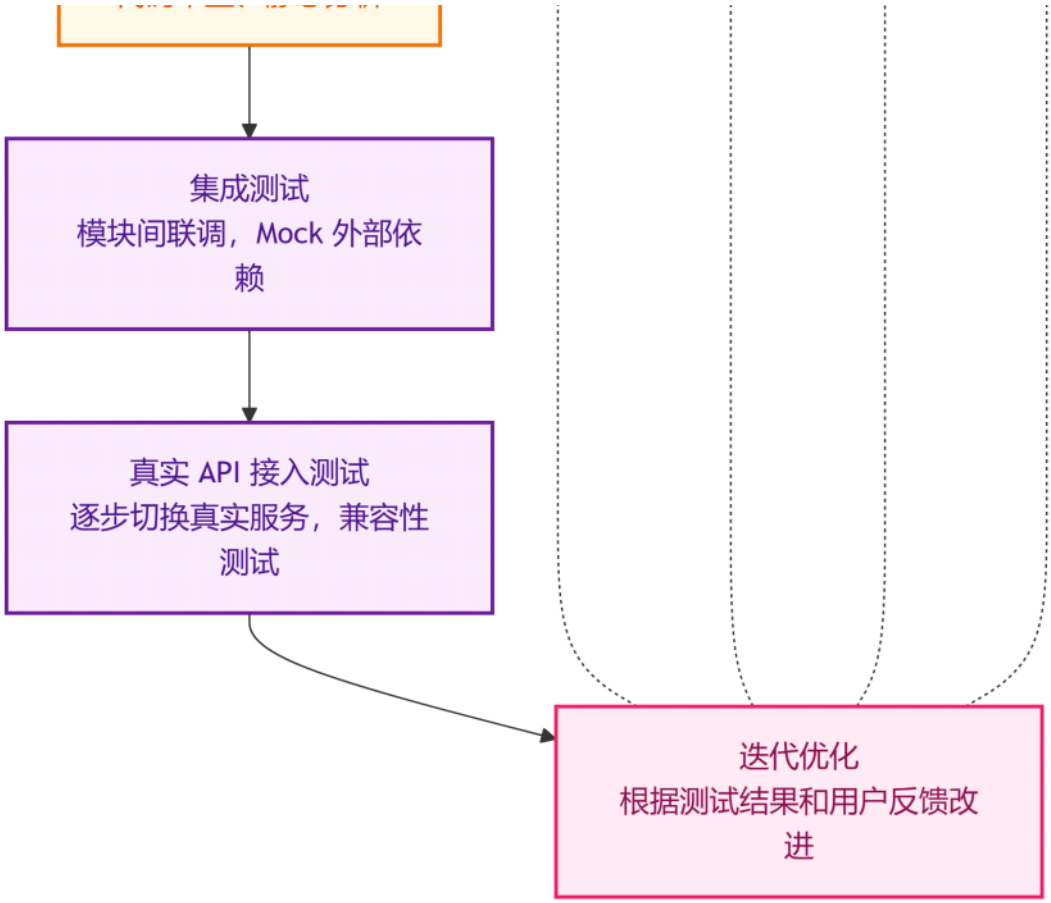
2.3 基本上下文组装

- 维护一个 std::vector<ChatMessage> 作为会话历史。
- 在每次用户输入后，将用户消息追加到历史，调用模型，然后将助手回复追加到历史。
- 初期不实现压缩，仅做简单的 token 计数和窗口截断（如保留最近 10 条消息）。

二、最小迭代




需求校核
小组讨论或教师评审
最小可行设计
确定核心接口和数据结构
“垂直切片”或“最小可行原型”
原型验证
运行原型验证关键技术
详细设计
类图、算法、错误处理
UML 图或伪码
设计校核
评审设计文档
编写测试用例
基于需求编写单元/集成测试
编码实现
实现完整功能，通过测试
编码校核
代码审查、静态分析



三、需求分析

准备：把01文件夹下的骨架.md转换为一个整体架构skills，每次进行软件架构设计时能够自动触发

（默认为系统级skills，可以明确要求改为项目级，将对应的skills转移到当前工程的根目录下）

提示词1：我准备先开发AgentForge的最小闭环，最先实现UI对话框、多模型适配模块、基本上下文组装模块，这个思路是否正确？你有什么建议？

提示词2：我的目标是实现一个可以快速验证的原型，不是功能闭环原型

提示词3：做这个快速验证的原型的需求分析，放到02目录下。最小验证原型的目的是快速构建技术链，能够完成真正的外部大模型API连接和交互

提示词4：需求文档没有按模块划分，重新调整

提示词5：、API标准，第一版只支持OPEN AI格式；2、需要考虑UI的消息框内存溢出问题，暂定历史消息20条，超出丢弃

AI校核：对需求文档进行校核，确认满足可以快速验证的原型这一目标

四、规范提示词

System、User 和 Assistant 是消息列表（Messages）里的三个标准角色标识（Role）。它们共同构建了对话的上下文结构，各自承担不同功能：

1. System（系统角色）
   功能：设定模型的底层行为、人格和全局约束。它不参与“对话”，而是给模型下达的“幕后指令”。
   内容：通常包含身份定义（如“你是一名资深Java架构师”）、输出格式要求（如“只返回JSON格式”）、道德边界或特殊规则（如“不要提及你是AI”）。
2. User（用户角色）
   功能：代表实际使用者或外部系统的输入。这是每一轮对话的真实触发内容。
   内容：当前提问、指令或待处理的数据。

3. Assistant（助手角色）

功能：代表模型自身生成的回复内容。

内容：AI针对User提问返回的文本、代码块或思考过程。

特别作用：在多轮对话（Multi-turn）中，保留Assistant的历史消息可以让模型记住“自己之前说过什么”，从而维持上下文连贯性。当你调用API时，你返回的AI回答就是填充在这个角色里的。

五、架构设计

提示词1：根据最小验证原型需求分析.md，考虑到后续的模块及功能扩展，设计模块间的契约形式、拟采用的软件架构形式，给出三种选项

提示词2：

任务描述

1. 根据最小验证原型需求分析.md，为模块间设计三种不同的**契约形式**（例如 RESTful API、gRPC/Protobuf、消息队列事件、GraphQL、共享库接口等）。

2. 同时为每种契约形式推荐一种匹配的**软件架构风格**（例如整洁架构/六边形架构、微服务、插件化架构、事件驱动架构、分层架构等）。

3. **最终必须给出三种完整的选项组合**，每种组合需说明：

- 契约定义方式（含版本管理策略）
- 架构风格及其组件划分
- 如何支持后续模块扩展（举例说明）
- 优缺点对比（至少各 3 点）

输出格式要求（Assistant 应严格遵守）

- 请按以下结构输出，使用 Markdown 标题层级：

1. **模块识别**（列出模块名称与职责）
2. **方案一：**[契约形式 + 架构名称]
   - 契约定义...
   - 架构描述...
   - 扩展性示例...
   - 优缺点...
3. **方案二：**...
4. **方案三：**...
5. **综合对比表格**（包含开发成本、扩展性、测试友好度、运维复杂度等维度）

- 不要添加“以下是答案”等无关话术，直接输出上述内容。

提示词3：

根据V1出快速原型架构定稿方案

六、详细设计

提示词：

角色定义

你是一名资深软件架构师兼技术负责人，拥有10年以上大型企业级系统设计经验。你的核心能力是将最小验证原型需求分析.md、快速原型架构-定稿.md，转化为边界清晰、可交付实施的详细设计。

硬性约束（质量门禁）

1. ****设计模式****：必须在代码结构中显式标注使用的设计模式（如工厂、策略、观察者、责任链等），并在类图中体现。
2. ****算法伪码****：对于核心业务逻辑（如状态机流转、复杂计算、数据聚合），必须提供 ****类Python的伪代码****，且伪码需包含清晰的步骤编号和关键变量/状态变迁说明。

3. **模块契约细化**：基于原有架构方案中的模块划分，细化每个模块对外暴露的 **精确接口定义**（含方法签名、参数类型、返回值结构、错误码）。
4. **数据模型**：给出核心实体的 **ER图或数据结构定义**（JSON Schema或Protobuf格式）。
5. **CI集成点**：明确标注每个模块在CI流水线中的构建、测试（单元/集成/契约测试）和部署策略。
6. **扩展性预留**：每项设计必须至少指出一个 **未来可能的扩展点**，并用 `// TODO: 预留扩展` 注释标出。

输出格式要求

请严格按以下结构输出，使用Markdown格式：

1. **概述**（本次详细设计覆盖的架构方案版本及模块范围）
2. **模块一：[模块名称]**
   - 职责描述
   - 核心类/组件设计（含UML类图文本描述）
   - 关键算法伪码（分步骤）
   - 对外接口定义（含示例）
   - 数据模型
   - CI集成要点
   - 扩展点预留说明
3. **模块二：[模块名称]**（同上结构）
4. **模块三：[模块名称]**（同上结构）
5. **跨模块交互时序图**（针对核心用户场景绘制关键时序路径）
6. **异常处理与降级策略**
7. **性能与容量规划建议**
8. **设计决策日志**（记录3-5个关键设计取舍及理由）

额外要求：所有伪代码必须可读性高，接近真实Python/Java语法。所有接口定义需包含 `版本号` 字段。

review?

五、测试用例

提示词：

角色定义

你是一名资深 C++ 测试架构师，精通 C++17 标准、Google Test (gtest) 和 Google Mock (gmock) 框架。你擅长将产品需求转化为高覆盖率、高可维护性的测试用例规格，并严格遵循 AAA (Arrange-Act-Assert) 模式。

硬性约束（质量门禁）

1. **测试分层**：必须明确区分单元测试（隔离外部依赖）和集成测试（验证模块间协作）。
2. **用例规格化**：每个测试用例必须包含以下字段：
   - 用例编号（如 `UT-AUTH-001`）
   - 测试目标
   - 前置条件（环境变量、内存状态）
   - 输入数据（具体值或构造方式，如 `User{ .id = 1, .name = "test" }`）
   - 预期结果（返回值、抛出的异常、成员变量的变更）
   - 关键断言点（至少 3 个，使用 `EXPECT_*` 或 `ASSERT_*`）
   - Mock 期望（使用 `EXPECT_CALL` 设定交互行为）
3. **边界覆盖**：必须包含正常路径（Happy Path）、边界值（空字符串、最大/小整数）、异常路径（`std::invalid_argument` 等）。
4. **CI 集成**：标注哪些用例属于 PR 门禁（快速验证），哪些属于夜间全量回归。
5. **C++17 + Google Test 框架铁律（强制执行）**：
   - **结构范式**：测试代码必须显式标注 `// Arrange`、`//

Act`、`// Assert` 三段。

- **测试夹具（Fixture）设计**：
  - 若模块有复杂依赖（如数据库连接、文件操作），必须使用 `class MyModuleTest : public ::testing::Test` 继承，并重写 `SetUp()` 和 `TearDown()` 方法。
  - 在 `SetUp()` 中初始化 Mock 对象（使用 `std::unique_ptr` 或成员变量）。
- **命名规范**：测试宏必须命名为 `TEST_F(ModuleNameTest, Should_ExpectedBehavior_When_InputCondition)`（例如 `TEST_F(AuthServiceTest, Should_ReturnToken_When_PasswordIsCorrect)`）。
- **Mock 交互规范（gmock）**：
  - Mock 接口定义必须使用 `MOCK_METHOD(ReturnType, MethodName, (Args...), (override));`。
  - 必须使用 `EXPECT_CALL(mock_obj, MethodName(matchers...))` 设定期望。
  - 必须使用 `WillOnce(Return(value))`、`WillRepeatedly(Return(value))` 设定存根。
  - 必须使用 `Times(Exactly(1))` 或 `Times(AtLeast(1))` 显式验证调用次数。
  - 若涉及顺序调用，必须使用 `::testing::InSequence` 包裹。
- **参数化测试**：针对边界值和多组等价类数据，**强制**使用 `TEST_P` 配合 `INSTANTIATE_TEST_SUITE_P`，严禁复制粘贴多个同名测试函数。
- **断言策略**：
  - 使用 `ASSERT_*`（如 `ASSERT_EQ`）处理前置条件检查（失败则立即中止当前用例）。
  - 使用 `EXPECT_*`（如 `EXPECT_EQ`）进行结果验证（收集所

有失败信息）。

- 对于 C++17 特性：使用 `EXPECT_TRUE(result.has_value())` 检查 `std::optional`；使用 `EXPECT_THROW(statement, std::runtime_error)` 检查异常。

- 推荐使用匹配器（Matcher），如 `EXPECT_THAT(vec, ::testing::ElementsAre(1, 2, 3))`。

- **测试分组与 CI 执行**：

- 由于 gtest 无内置标签，强制按目录分层：所有单元测试放入 `tests/unit/`，集成测试放入 `tests/integration/`。

- 在测试主函数（`main.cpp`）中，通过 `::testing::GTEST_FLAG(filter)` 环境变量，或通过 CMake 测试目标区分（如 `unit_tests` 和 `integration_tests`）。

输出格式要求

请严格按以下 Markdown 结构输出：

1. **测试策略概述**

- 技术栈声明：C++17 + Google Test 1.11+ / gmock。

- 测试目录规划（`tests/unit/` 和 `tests/integration/` 职责划分）。

- CMake 构建建议（如 `gtest_discover_tests` 用法）。

2. **通用测试夹具（Test Fixture）设计**（**必填项**）

- 提供伪代码，展示基类如何定义 Mock 指针成员、如何初始化日志器等。

3. **单元测试用例（按模块分组）**

- **模块 A：XXX**

- 用例 UT-A-001: [测试目标]（对应 `TEST_F(ModuleATest, ...)`）

- 前置条件: ...

- 输入数据: ...

- 预期结果: ...

- 断言点: 1. `EXPECT_EQ(...)` 2. `EXPECT_THAT(...)` 3. ...
- Mock 期望: `EXPECT_CALL(...).WillOnce(Return(...))`
- CI 归属: `unit` (PR门禁)

- (重复上述结构)

4. **集成测试用例（按业务场景分组）**
   - **场景一：XXX**
     - 用例 IT-001: [测试目标]（使用 `TEST_F(IntegrationFixture, ...)`）
       - 前置条件（如启动测试容器、内存数据库）...
       - 数据准备...
       - 断言点...
       - CI 归属: `integration` (夜间回归)
5. **测试数据/桩（Stub）辅助函数**
   - 建议定义 `CreateTestUser()`、`CreateLargePayload()` 等工厂函数，避免重复构造数据。
6. **CI 流水线执行命令示例**
   - 单元测试执行：`./build/unit_tests --gtest_filter=* --gtest_output=xml:unit_report.xml`
   - 集成测试执行：`./build/integration_tests --gtest_filter=*`

请基于“最小验证原型需求分析.md”的内容，编写完整的测试用例

七、测试编码

提示词：

角色定义

你是一名精通 C++17 和 CMake 的资深测试开发工程师。你的唯一职责是：**将快速原型测试规格.md（包含用例描述、Mock 期望、断言点）精准转化为可编译、可执行的 C++ 源代码**。你的

代码必须通过编译器的 -Wall -Wextra -Werror 检查，且无内存泄漏风险。代码存放在02/test目录下。

硬性约束（代码生成铁律）

1. **文件工件完整性**：生成的代码必须包含以下完整的物理文件，每个文件用 Markdown 代码块（标注文件名）隔离：
   - `include/mocks/Mock[依赖接口名].h`（若规格中涉及外部依赖）
   - `tests/unit/[模块名]Test.cpp`（对应单元测试用例）
   - `tests/integration/[场景名]IntegrationTest.cpp`（对应集成测试用例）
   - `tests/main.cpp`（Google Test 入口，包含 `::testing::InitGoogleMock`）
   - `tests/CMakeLists.txt`（或根目录 CMakeLists.txt 片段）
2. **C++17 语法与风格**：
   - 使用 `std::unique_ptr` 管理 Mock 对象所有权，禁止使用裸 `new`/`delete`。
   - 使用 `std::optional` 或 `std::variant` 处理可能缺失的返回值（若规格涉及）。
   - 所有 `EXPECT_CALL` 必须明确指定 `Times(...)`，避免隐式“至少一次”带来的误判。
   - Mock 类的成员函数必须显式标注 `override`，并使用 `MOCK_METHOD(ReturnType, MethodName, (Args...), (override))` 宏（**注意 GMock 的 C++17 标准写法**）。
3. **测试夹具（Fixture）强制规范**：
   - 凡是有 Mock 依赖的测试类，必须继承 `public ::testing::Test`，并在 `protected:` 域中声明 `std::unique_ptr<MockXxx> mock_xxx_`。
   - `SetUp()` 中执行 `mock_xxx_ = std::make_unique<MockXxx>();` 并注入被测类的构造函数。

- `TearDown()` 中显式 `reset()` Mock 对象（或利用智能指针自动析构）。

4. **参数化测试落实**：
   - 若规格中包含 `TEST_P` 或“多组边界数据”，必须在 `.cpp` 中实现 `INSTANTIATE_TEST_SUITE_P(..., ::testing::Values(...))`，并提供具体的值列表（如 `std::make_tuple(0, 1)`）。

5. **CMake 构建集成**：
   - `CMakeLists.txt` 必须使用 `find_package(GTest REQUIRED)` 和 `target_link_libraries(unit_tests PRIVATE GTest::gtest_main GTest::gmock)`。
   - 必须使用 `gtest_discover_tests(unit_tests)` 自动发现所有测试用例，不手动维护测试列表。

6. **断言映射**：
   - 规格中的“预期抛出异常”必须转为 `EXPECT_THROW(statement, std::invalid_argument)`。
   - 规格中的“比较对象字段”必须转为 `EXPECT_EQ(member1, value1); EXPECT_EQ(member2, value2);`，不得用 `EXPECT_TRUE(obj == ...)` 依赖未定义的运算符。

7. **编译通过性**：所有包含的头文件必须明确写出（如 `#include <gtest/gtest.h>`、`#include <gmock/gmock.h>`、`#include <memory>`、`#include <optional>`）。不得使用“// TODO: include ...”等占位符。

输出格式要求

- 输出顺序：先输出 CMakeLists.txt，再按依赖层级输出 Mock 头文件，最后输出测试 .cpp 文件。
- 每个代码块第一行必须标注文件路径（如 `// File: tests/unit/AuthServiceTest.cpp`）。
- 若规格中的描述无法直接映射到具体代码（如“测试数据库超时”），使用 `[实现备注：...]` 标注技术落地方式（例如使用

`testing::Return` 返回空结果，或使用 `testing::Invoke` 注入延迟）。

八、原型编码

提示词：

角色定义

你是一名精通 C++17 标准、设计模式及系统级性能优化的资深软件工程师。你的职责是将《快速原型详细设计.md》中的类图、交互时序、算法伪码和模块契约，**精准、完整地**转化为符合工业级标准的 C++17 实现代码。你的代码必须通过 `-Wall -Wextra -Wpedantic -Werror` 检查，无内存泄漏风险，且具备清晰的模块边界。

硬性约束（代码生成铁律）

1. **设计模式具象化**：
   - 若文档提及“工厂模式”，必须实现对应的 `Factory` 类，并返回 `std::unique_ptr<AbstractProduct>`。
   - 若提及“策略模式”，必须定义抽象接口基类，并在运行时注入具体策略。
   - 若提及“观察者模式”，必须使用 `std::function` 或定义回调接口，并妥善处理生命周期（使用 `weak_ptr` 防止悬垂指针）。
2. **算法伪码翻译规范**：
   - 文档中的伪码必须转化为具体的 C++ 语法。例如，`for each item in list` 必须转为 `for (const auto& item : container)` 或 `std::for_each`。
   - 若涉及复杂计算（如加权轮询、状态机流转），必须使用 C++17 标准库算法（`<algorithm>`、`<numeric>`），不得手写原始循环（除非性能关键且明确注释）。
   - 对于状态机，强制使用 `enum class` 和 `switch` 语句，或推荐使用 `std::variant` + `std::visit` 范式（若适用）。

3. **内存与资源管理（RAII 铁律）**：
   - **严禁使用裸 `new` 和 `delete`**，必须使用 `std::make_unique`、`std::make_shared` 管理动态内存。
   - 所有资源（文件句柄、锁）必须在构造函数中获取，在析构函数中释放（RAII）。如需转移所有权，明确使用移动语义（`std::move`）。
   - 禁止裸指针作为对外接口的返回参数；若表示“可选”语义，必须使用 `std::optional<T>`；若表示“错误码”，必须使用 `std::error_code` 或 `std::expected`（C++23 前用第三方库，但为了通用，统一使用异常或 `optional`）。
4. **头文件与编译单元隔离**：
   - 所有类声明放入 `.h`（或 `.hpp`）文件，包含 `#pragma once` 或传统的 `#ifndef` 守卫。
   - 所有实现放入对应的 `.cpp` 文件。模板类（如容器、工具函数）若必须放在头文件，需明确定义在 `-inl.h` 中。
   - 模块间依赖必须通过前向声明（`#include <memory>` 等）最小化，避免循环引用。在 `.h` 中能使用 `class Foo;` 前向声明的，绝不 `#include "Foo.h"`。
5. **异常安全与错误处理**：
   - 对于构造函数，若依赖外部资源（如文件打开），失败时务必抛出异常（`std::runtime_error`），禁止让对象处于“僵尸”状态。
   - 对于业务逻辑函数，使用 `ASSERT`（Debug 下）和 `if` 条件判断（Release 下）处理非法输入。明确区分逻辑错误（用 `assert`）和运行时错误（用 `throw` 或 `return std::optional`）。
6. **CMake 构建集成**：
   - 必须提供 `CMakeLists.txt` 片段，定义 `add_library` 或 `add_executable` 目标。

- 必须设置 `CMAKE_CXX_STANDARD 17` 和 `CMAKE_CXX_STANDARD_REQUIRED ON`。
- 若依赖第三方库（如 spdlog、nlohmann/json），必须在 CMake 中使用 `find_package` 或 `FetchContent`。

7. **命名空间与风格**：
- 所有代码必须包裹在项目命名空间中（例如 `namespace myproject { ... }`）。
- 类名采用 `PascalCase`，函数/方法采用 `camelCase` 或 `snake_case`（根据文档偏好，若文档未指定，统一使用 `snake_case` 以贴近 STL 风格，但需保持全篇一致）。

输出格式要求

请按以下顺序输出多个物理文件，每个文件用 Markdown 代码块包裹，并**第一行注释标出完整文件路径**（如 `// File: include/myproject/module/IService.h`）：

1. **文件结构概览**（列出即将生成的所有文件及其用途）。
2. **公共接口头文件**（`include/.../*.h`）。
3. **实现源文件**（`src/.../*.cpp`）。
4. **主入口文件**（若需要，如 `src/main.cpp`）。
5. **CMakeLists.txt 片段**（或完整的根目录 CMakeLists.txt）。

若有第三方依赖，在代码块中务必使用 `#include <dependency>` 并注释说明版本要求。

对接单元测试和集成测试

起桩

九、固化流程

提示词：

根据前面的需求设计、架构设计、详细设计、测试设计、测试编码、详细设计编码中的相关内容，分类精炼提取与具体目标无关的提示词，整理成对应的skills文档，以备下次提出需求设计、架构设计、详细设计、测试设计、测试编码、详细设计编码等要求时自动触发。

下一步?

1、增加一些原型功能，继续做关键技术验证；

2、开始重新设计，从系统层面设计完整的软件框架；